
Taller 2

Búsqueda de conexiones en una red social

Implementación y análisis de BFS y DFS

Asignatura: Inteligencia Artificial
Tema: Búsqueda no informada
Docente: Joaquín F. Sánchez
Modalidad: Trabajo en grupos
Herramienta: Python 3

Tema

Implementación y análisis de algoritmos de búsqueda no informada para encontrar rutas de conexión entre usuarios de una red social.

Situación problema

Una red social puede representarse mediante un grafo en el que cada usuario corresponde a un nodo y cada relación de amistad corresponde a una arista. La plataforma desea incorporar una herramienta que determine si dos usuarios están relacionados directa o indirectamente y que muestre la secuencia de contactos que permite conectarlos.

Por ejemplo, la ruta

Ana → Bruno → Diana → Helena

indica que Ana puede conectarse con Helena mediante Bruno y Diana, con tres grados de separación.

Propósito

Aplicar los fundamentos de búsqueda no informada en un problema representado mediante grafos, analizando cómo BFS y DFS recorren una red social y encuentran rutas entre diferentes usuarios.

Red social de referencia

La red se considera no dirigida. Si Ana está conectada con Bruno, Bruno también debe estar conectado con Ana.

```
red_social = {  
    "Ana": ["Bruno", "Carla", "Elena"],  
    "Bruno": ["Ana", "Diana", "Felipe"],  
    "Carla": ["Ana", "Felipe", "Gabriel"],  
    "Diana": ["Bruno", "Helena"],  
    "Elena": ["Ana", "Isabel"],  
    "Felipe": ["Bruno", "Carla", "Helena"],  
    "Gabriel": ["Carla", "Jorge"],  
    "Helena": ["Diana", "Felipe", "Karen"],  
    "Isabel": ["Elena", "Karen"],  
    "Jorge": ["Gabriel", "Karen"],  
    "Karen": ["Helena", "Isabel", "Jorge"]  
}
```

Punto 1. Formulación del problema

Analice la búsqueda de una conexión entre Ana y Karen. Complete la tabla.

Componente	Representación en la red social
Estado inicial	
Estado objetivo	
Estados	
Acciones	
Modelo de transición	
Prueba de objetivo	
Costo del camino	
Solución	

Responda:

1. ¿Qué representa un nodo?
2. ¿Qué representa una arista?
3. ¿Qué significa expandir un usuario?
4. ¿Qué información debe contener un nodo de búsqueda?
5. ¿Cómo se determina si se alcanzó el objetivo?
6. ¿Qué representan los grados de separación?

Punto 2. Construcción y análisis del grafo

A partir del diccionario `red_social`:

1. Dibuje el grafo correspondiente.
2. Identifique los vecinos de cada usuario.
3. Determine el grado de cada nodo.
4. Identifique el usuario con mayor cantidad de conexiones.
5. Determine si existen usuarios aislados.
6. Verifique que todas las conexiones sean bidireccionales.

Usuario	Vecinos	Grado
Ana		

Usuario	Vecinos	Grado
Bruno		
Carla		
Diana		
Elena		
Felipe		
Gabriel		
Helena		
Isabel		
Jorge		
Karen		

Pregunta: ¿Un usuario con muchas conexiones garantiza que todas las rutas que pasan por él sean las más cortas? Justifique.

Punto 3. Ejecución manual de BFS

Utilice BFS para buscar una ruta desde **Ana** hasta **Karen**. El orden de los vecinos debe ser el definido en el diccionario.

1. Agregue a Ana en una cola FIFO.
2. Registre a Ana como visitada.
3. Extraiga el primer usuario de la cola.
4. Compruebe si corresponde al objetivo.
5. Agregue sus vecinos no visitados.
6. Registre el padre de cada usuario.
7. Continúe hasta encontrar a Karen.

N.º	Extraído	Agregados	Cola	Visitados
1				
2				

N.º	Extraído	Agregados	Cola	Visitados
3				
4				
5				
6				
7				
8				

Indique el orden de expansión, la ruta, los grados de separación, la cantidad de nodos expandidos y la frontera máxima.

Punto 4. Ejecución manual de DFS

Repita la búsqueda desde Ana hasta Karen utilizando una pila LIFO. Registre el orden de inserción de los vecinos.

N.º	Extraído	Agregados	Pila	Visitados
1				
2				
3				
4				
5				
6				
7				
8				

Compare la ruta obtenida con la encontrada mediante BFS.

Punto 5. Implementación de BFS

Implemente la función:

```
def buscar_conexion_bfs(red, origen, destino):  
    pass
```

La función debe retornar una estructura como:

```
{  
    "encontrado": True,  
    "ruta": ["Ana", "Elena", "Isabel", "Karen"],  
    "grados_separacion": 3,  
    "orden_expansion": [],  
    "nodos_expandidos": 0,  
    "nodos_generados": 0,  
    "frontera_maxima": 0  
}
```

Complete el siguiente código:

```
from collections import deque  
  
def reconstruir_ruta(padres, destino):  
    ruta = []  
    actual = destino  
  
    while actual is not None:  
        ruta.append(actual)  
        actual = padres[actual]  
  
    ruta.reverse()  
    return ruta  
  
def buscar_conexion_bfs(red, origen, destino):  
    if origen not in red or destino not in red:  
        return {  
            "encontrado": False,  
            "mensaje": "El usuario no existe en la red."  
        }  
  
    frontera = deque([origen])  
    visitados = {origen}  
    padres = {origen: None}  
  
    orden_expansion = []  
    nodos_generados = 1  
    frontera_maxima = 1  
  
    while frontera:  
        actual = frontera.popleft()  
        orden_expansion.append(actual)  
  
        if actual == destino:
```

```

        ruta = reconstruir_ruta(padres, destino)
        return {
            "encontrado": True,
            "ruta": ruta,
            "grados_separacion": len(ruta) - 1,
            "orden_expansion": orden_expansion,
            "nodos_expandidos": len(orden_expansion),
            "nodos_generados": nodos_generados,
            "frontera_maxima": frontera_maxima
        }

    for vecino in red[actual]:
        # Completar el control de visitados,
        # el padre y la insercion en la cola.
        pass

    return {
        "encontrado": False,
        "ruta": [],
        "grados_separacion": None,
        "orden_expansion": orden_expansion,
        "nodos_expandidos": len(orden_expansion),
        "nodos_generados": nodos_generados,
        "frontera_maxima": frontera_maxima
    }

```

Punto 6. Implementación de DFS

Implemente:

```

def buscar_conexion_dfs(
    red,
    origen,
    destino,
    limite_profundidad=10
):
    if origen not in red or destino not in red:
        return {
            "encontrado": False,
            "mensaje": "El usuario no existe en la red."
        }

    frontera = [(origen, 0)]
    visitados = set()
    padres = {origen: None}

    orden_expansion = []
    nodos_generados = 1
    frontera_maxima = 1

    while frontera:
        actual, profundidad = frontera.pop()

```

```

    if actual in visitados:
        continue

    visitados.add(actual)
    orden_expansion.append(actual)

    if actual == destino:
        ruta = reconstruir_ruta(padres, destino)
        return {
            "encontrado": True,
            "ruta": ruta,
            "grados_separacion": len(ruta) - 1,
            "orden_expansion": orden_expansion,
            "nodos_expandidos": len(orden_expansion),
            "nodos_generados": nodos_generados,
            "frontera_maxima": frontera_maxima
        }

    if profundidad >= limite_profundidad:
        continue

    # Completar la generacion e insercion
    # de vecinos en la pila LIFO.

    return {
        "encontrado": False,
        "ruta": [],
        "grados_separacion": None,
        "orden_expansion": orden_expansion,
        "nodos_expandidos": len(orden_expansion),
        "nodos_generados": nodos_generados,
        "frontera_maxima": frontera_maxima
    }

```

La implementación debe incluir orden de vecinos, control de profundidad, padres y métricas.

Punto 7. Pruebas de funcionamiento

Ejecute BFS y DFS para las siguientes consultas:

Prueba	Origen	Destino
1	Ana	Karen
2	Bruno	Jorge
3	Diana	Isabel
4	Gabriel	Elena
5	Felipe	Karen
6	Helena	Ana

P.	Alg.	Ruta	Grados	Exp.	Gen.	F. máx.
1	BFS					
1	DFS					
2	BFS					
2	DFS					
3	BFS					
3	DFS					
4	BFS					
4	DFS					
5	BFS					
5	DFS					
6	BFS					
6	DFS					

Punto 8. Comparación de BFS y DFS

Responda con base en los resultados:

1. ¿BFS y DFS encontraron siempre la misma ruta?
2. ¿Cuál encontró menos grados de separación?
3. ¿Qué algoritmo expandió menos usuarios?
4. ¿Cuál presentó la frontera más grande?
5. ¿Cómo influyó el orden de los vecinos?
6. ¿Qué sucede si no se usa un conjunto de visitados?
7. ¿Por qué BFS garantiza la ruta con menos conexiones?
8. ¿DFS puede encontrar primero una ruta más larga?
9. ¿Cómo afecta el límite de profundidad?
10. ¿Qué algoritmo recomendaría para calcular grados de separación?

Punto 9. Usuarios sin conexión

Agregue un componente separado:

```
red_social["Laura"] = ["Mateo"]  
red_social["Mateo"] = ["Laura"]
```

Ejecute:

```
buscar_conexion_bfs(red_social, "Ana", "Laura")  
buscar_conexion_dfs(red_social, "Ana", "Laura", 10)
```

Analice cómo se determina que no existe conexión, qué sucede con la frontera, cuántos usuarios se visitan y qué mensaje debe presentar el programa.

Punto 10. Modificación de la red

Cada grupo debe agregar al menos cinco usuarios ficticios y ocho conexiones. La modificación debe cumplir:

- la red debe seguir siendo no dirigida;
- no se permiten conexiones duplicadas;
- un usuario no puede conectarse consigo mismo;
- debe existir al menos una ruta alternativa;
- debe existir un usuario con una única conexión.

Después de modificarla, dibuje el grafo, seleccione tres pares de usuarios, ejecute BFS y DFS, compare las rutas y explique los cambios observados.

Punto 11. Mejora del sistema

Seleccione una mejora:

- profundización iterativa;
- menú para agregar usuarios y conexiones;
- lectura de la red desde JSON;
- exportación de resultados a CSV;
- visualización mediante NetworkX;
- interfaz gráfica;
- identificación de componentes desconectados;

- comparación simultánea de BFS y DFS;
- medición del tiempo de ejecución.

Incluya descripción, justificación, código, evidencia y análisis.

Entregables

1. Código fuente en Python.
2. Representación gráfica de la red.
3. Tablas de ejecución manual.
4. Implementaciones de BFS y DFS.
5. Resultados de las seis pruebas.
6. Comparación experimental.
7. Respuestas de análisis.
8. Mejora implementada.
9. Evidencias de ejecución.
10. Archivo README.md con instrucciones.

Estructura sugerida

```
taller_2_red_social/  
|-- main.py  
|-- busquedas.py  
|-- red_social.py  
|-- datos/  
|   '-- red_social.json  
|-- evidencias/  
|-- README.md  
'-- informe.pdf
```

Criterios de valoración

Criterio	Porcentaje
Formulación del problema como grafo	15%
Ejecución manual de BFS y DFS	15%
Implementación de BFS	20%
Implementación de DFS	20%
Experimentación y métricas	15%
Análisis comparativo	10%
Documentación y presentación	5%
Total	100%